image_multi_view main 브랜치
코드 분석 및 Pinpoint 특허 팩트체크 보고서

compareX / image_multi_view main branch 직접 분석 기반

작성 목적

GitHub main 브랜치의 실제 코드 구조와 동작 로직을 직접 확인하고, 기존 특허 정리서의 진술이 어디까지 코드로 확인되는지 팩트체크하며, 특허 서술에 반영해야 할 강화 포인트와 표현 조정 포인트를 함께 정리하기 위함입니다.

분석 대상 파일

README.md, package.json, src/App.tsx, src/store.ts, src/modes/PinpointMode.tsx, src/modes/CompareMode.tsx, src/modes/AnalysisMode.tsx, src/components/ImageCanvas.tsx, LayoutGridSelector.tsx, DraggableViewer.tsx, usage logging / Electron 관련 파일

기준 문서

이전 작성본: compareX_Pinpoint_patent_brief_ko.docx

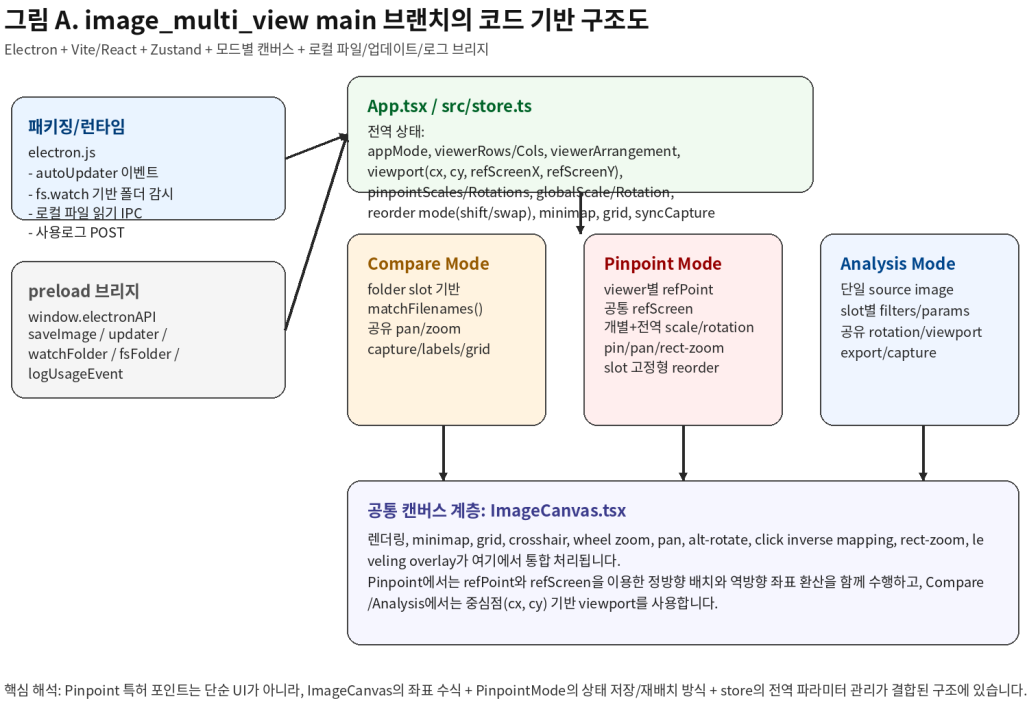

그림 A. 코드 기반 아키텍처 개요

0. 핵심 요약

결론부터 말씀드리면, 기존 특허 정리서의 큰 방향은 맞습니다. 다만 GitHub main 브랜치의 실제 코드를 대조해 보니, Pinpoint 의 핵심은 "창별 refPoint 저장 + 공통 refScreen 기반 정방향 배치 + 동일 파라미터를 쓰는 역방향 좌표 환산 + slot 고정형 재배치"로 더 선명하게 정리할 수 있었고, 일부 표현은 구현 범위에 맞게 조정해야 합니다.

- A 축(좌표 정합 엔진)은 코드상 매우 강하게 확인됩니다. ImageCanvas 는 per-viewer refPoint, viewport.refScreenX/refScreenY, local/global scale·rotation 을 결합하여 drawX/drawY 를 계산하고, 클릭 시에는 같은 파라미터 집합으로 screen→image 역변환을 수행합니다.
- B 축(운영 시스템)도 확인됩니다. App.tsx 에는 Pin/Pan 분리, Shift/Swap 재배치 모드, 공통 LayoutGridSelector 가 있고, PinpointMode 는 slot 고정 상태에서 image·scale·rotation·filter 상태를 함께 이동시키는 별도 재배치 로직을 구현합니다.
- 기존 문서에서 "shared viewport"라고 묶어 적었던 부분은 용어를 더 정확히 나누는 편이 좋습니다. Pinpoint 는 viewport.refScreenX/refScreenY 를 공유하고, Compare/Analysis 는 viewport.cx/cy 를 공유합니다.
- "자동 정합"이라는 표현은 현재 main 브랜치 코드와 맞지 않습니다. 현재 구현은 자동 특징점 정합이 아니라 사용자 주도형 인터랙티브 정밀 정합입니다.
- Analysis 모드는 README 의 표현과 달리 현재 main 브랜치에서는 slot 별 독립 rotation 이 아니라 공유 analysisRotation 을 사용합니다. 따라서 특허 핵심 논리는 Pinpoint 중심으로 두는 편이 정확합니다.
- README 는 local-first, pixel stays on client 를 강조하지만, 실제 코드에는 usage logging endpoint 와 auto-update feed 설정이 있으므로 "이미지 처리 파이프라인은 로컬 중심" 정도로 쓰는 편이 사실에 더 가깝습니다.

1. 분석 범위와 방법

본 보고서는 GitHub 공개 저장소 MouseBall54/image_multi_view 의 main 브랜치를 기준으로 하였고, README 와 핵심 TypeScript/JavaScript 파일을 직접 열람하여 기능 설명과 구현 로직을 대조했습니다. 특히 특허와 직접 연결되는 Pinpoint 관련 좌표 모델, 입력 처리, 재배치, 상태 저장 구조를 우선순위로 분석했습니다.

파일	대략 라인 수	주요 확인 포인트
README.md	130	제품 설명, 모드 구성, 24 viewer, local-first 설명, Typical workflow
package.json	130	패키지명 compareX, Electron/Vite/React/Zustand/OpenCV/UTIF 의존성, usageLogging·deployTargets
src/App.tsx	1,499	모드 전환, LayoutGridSelector, Pin/Pan, Shift/Swap, 글로벌 회전·배율 UI
src/store.ts	1,167	전역 상태 구조, viewport/refScreen, pinpointGlobalScale/Rotation, reorder mode, leveling
src/modes/PinpointMode.tsx	1,170	viewer 별 refPoint 보유, loadFileToViewer, slot 고정형 재배치, 동일 파일 다중 배치 상태
src/components/ImageCanvas.tsx	1,823	정방향 배치 수식, inverse mapping, pan/zoom/rotate, rect-zoom, leveling, minimap
src/modes/CompareMode.tsx	689	matchFilenames 기반 compare workflow
src/modes/AnalysisMode.tsx	766	단일 source + slot 별 filter, 공유 analysisRotation
electron.js / preload.js	739 / 88	auto-updater, fs.watch 폴더 감시, IPC, usage logging

분석 기준은 “화면상 기능이 보인다” 수준이 아니라, 어떤 상태가 어디에 저장되고 어떤 이벤트에서 어떤 값이 갱신되는지를 추적하는 방식으로 잡았습니다. 그래서 특허 문장도 UI 마케팅 문구보다 코드가 실제로 보장하는 구조를 기준으로 다시 정리했습니다.

2. 리포지토리와 실행 구조

2.1 런타임/프레임워크 구조

package.json 기준 이 프로젝트의 패키지명은 compareX 이고, Electron + Vite + React + Zustand 를 중심으로 구성되어 있습니다. 이미지 처리를 위해 opencv-ts, TIFF 처리를 위해 tiff 및 utif 가 포함되어 있으며, Electron desktop packaging 과 auto-update 를 위한 electron-updater 도 포함되어 있습니다.

- 렌더러 계층: React 컴포넌트(App.tsx, modes/*, components/*)
- 전역 상태: Zustand 기반 src/store.ts
- 데스크톱 브리지: preload.js → window.electronAPI
- 메인 프로세스: electron.js (업데이트, 파일 I/O, 폴더 감시, 저장, 로깅)
- 이미지 캔버스 코어: ImageCanvas.tsx

2.2 App shell 의 역할

App.tsx 는 단순 라우터가 아니라 모드 전환, 레이아웃, 공통 툴바, 전역 회전/배율 제어, Sync, Capture, minimap/grid 관련 설정을 모두 묶는 상위 orchestration 계층입니다. 여기서 Pinpoint 전용 Mouse 모드(Pin/Pan)와 재배치 모드(Shift/Swap)가 노출됩니다.

2.3 store 의 역할

src/store.ts 는 appMode, viewerRows/Cols, viewerArrangement, viewport, pinpointScales, pinpointGlobalScale, pinpointRotations, pinpointGlobalRotation, rect-zoom target, minimap/grid 상태, analysis/compare rotation, leveling state 등을 보관합니다. 즉 Pinpoint 특허 서술에서 “전역 상태 저장부”에 해당하는 상당 부분은 실제로 store.ts 에 존재합니다.

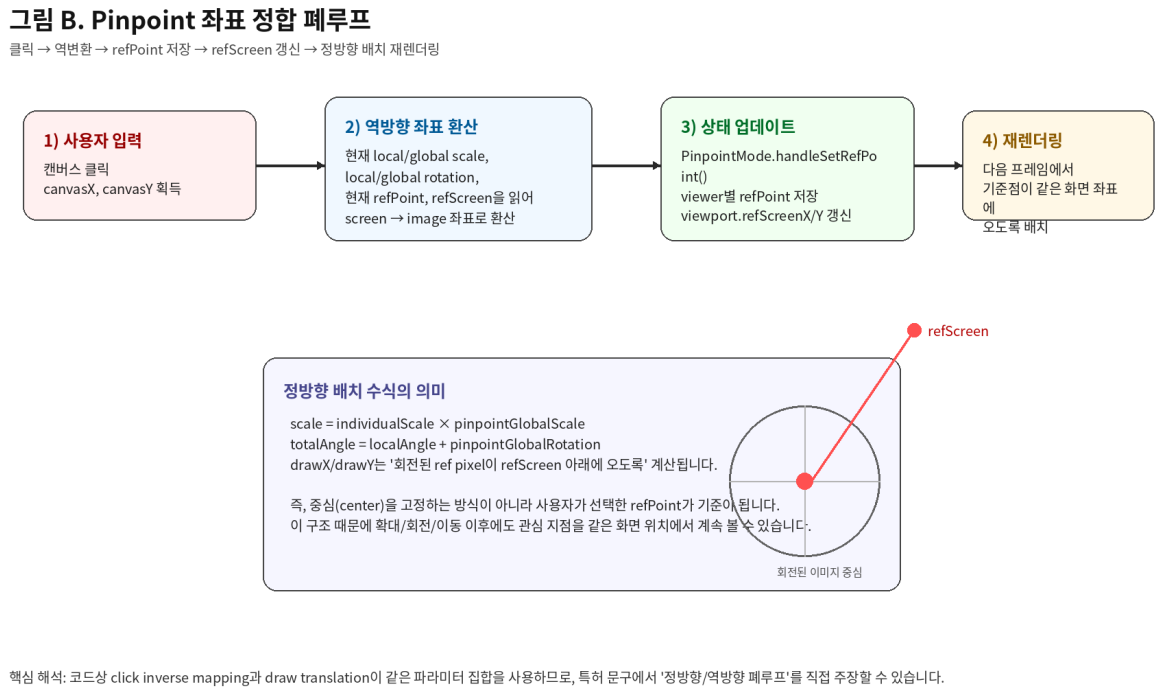

그림 B. Pinpoint 좌표 정합 폐루프

3. 모드별 동작 방식 분석

3.1 Compare 모드

Compare 모드는 folder slot 기반의 다중 비교 모드입니다. activeKeys = viewerArrangement.compare.slice(0, numViewers)로 현재 레이아웃에 활성화된 compare slot 을 구성하고, matchFilenames(activeFolders, stripExt)로 파일명을 맞춰 비교 대상을 생성합니다. 이 모드는 shared pan/zoom, grid/minimap, capture, toggle modal, reorderViewers 기반 위치 재정렬을 중심으로 동작합니다.

쉬운 해설: Compare 는 "여러 폴더에서 같은 이름의 파일을 찾아 한 번에 비교하는 모드"라고 보시면 됩니다. 특허의 핵심은 아니지만, Pinpoint 로 넘어가기 전 대량 훑어보기의 전 단계로 해석할 수 있습니다.

3.2 Pinpoint 모드

Pinpoint 모드는 이 저장소에서 특허 포인트와 가장 직접적으로 연결되는 모드입니다. loadFileToViewer()는 특정 viewer(targetKey)에 File 객체와 refPoint, sourceKey 를 묶어 넣고, handleSetRefPoint()는 특정 viewer 의 refPoint 를 갱신하면서 동시에 viewport.refScreenX/refScreenY 를 갱신합니다. 이후 ImageCanvas 는 viewer 별 refPoint 와 공통 refScreen 을 이용해 draw translation 을 계산합니다.

쉬운 해설: Pinpoint 는 "각 창에서 기준점을 찍고, 그 기준점이 화면에서 계속 같은 위치에 있게 만드는 모드"입니다. 지금 특허 문서는 사실상 이 모드를 중심으로 써야 합니다.

3.3 Analysis 모드

Analysis 모드는 하나의 source image 를 여러 slot 에 복제하여 서로 다른 filter 결과를 나란히 보는 구조입니다. slot 별로 analysisFilters[i], analysisFilterParams[i]는 독립적이지만, rotation 은 현재 구현상 rotation={analysisRotation}으로 모든 slot 이 공유합니다. 따라서 README 의 "independent rotation and zoom" 문구는 현재 코드와 완전히 일치하지 않습니다.

쉬운 해설: Analysis 는 "한 장의 이미지를 여러 필터 버전으로 비교하는 모드"이고, 현재는 회전값을 각 칸마다 따로 주는 구조가 아니라 전체가 같이 돌아가는 구조입니다.

4. Pinpoint 좌표 모델의 실제 코드 구조

4.1 상태가 어디에 저장되는가

상태	실제 저장 위치	의미	특허 문장으로 일반화할 때 주의점
viewer 별 refPoint	PinpointMode 내부 state pinpointImages[targetKey].refPoint	각 viewer 가 어떤 이미지 지점을 기준으로 삼는지 저장	"창별 기준 참조점 저장부"로 일반화 가능하나, 현재 구현은 전역 store 가 아니라 모드 로컬 state 입니다.
공통 화면 기준점	store.viewport.refScreenX / refScreenY	화면상 기준점이 놓일 공통 위치	기술적으로는 "공통 화면 기준 좌표 관리부"가 맞습니다.
개별 배율	store.pinpointScales[key]	viewer 별 미세 scale 조정	viewer 별 local transform 으로 서술 가능
전역 배율	store.pinpointGlobalScale	전체 Pinpoint 뷰 공통 scale	global transform 으로 서술 가능
개별 회전	store.pinpointRotations[key]	viewer 별 미세 회전	viewer 별 local rotation 으로 서술 가능
전역 회전	store.pinpointGlobalRotation	전체 Pinpoint 뷰 공통 회전	global rotation 으로 서술 가능

핵심은 refPoint 는 viewer 별로 분리되어 있고, refScreen 은 화면 차원에서 공유된다는 점입니다. 바로 이 분리 때문에 "여러 창이 하나의 화면 좌표를 공유하면서도 각자 다른 관심점을 붙잡는 구조"가 성립합니다.

4.2 정방향 배치(Forward transform)

ImageCanvas.tsx 는 Pinpoint 모드에서 scale = individualScale × pinpointGlobalScale, totalAngle = localAngle + pinpointGlobalRotation 을 계산하고, refPoint 가 refScreen 아래에 오도록 x, y 를 직접 풉니다. 주석도 “Solve translation so that rotated ref pixel stays under refScreen”이라고 되어 있어, 이 수식이 우연한 구현이 아니라 의도된 좌표 모델임을 분명히 보여 줍니다.

기술적 서술 예시

정방향 배치 변환부는 표시창별 기준 참조점이 공통 화면 기준좌표에 대응되도록 총배율 및 총회전각을 반영하여 배치 좌표(drawX, drawY)를 산출한다.

쉬운 설명

“내가 찍은 점이 화면의 이 자리로 오게 그림을 어디에 그려야 하나?”를 매 프레임 다시 계산하는 구조입니다.

4.3 역방향 좌표 환산(Inverse mapping)

기존 특허 정리서에서 “정방향/역변환 폐루프”라고 표현했던 부분은 이번 코드 분석으로 명확히 확인되었습니다. ImageCanvas 의 click handler 는 현재 refPoint, refScreen, local/global scale, local/global rotation 을 다시 읽어 drawX/drawY 를 재계산한 뒤, inverse rotation 과 scale 역적용을 통해 canvas 좌표를 image 좌표로 환산합니다. 그리고 그 결과를 onSetRefPoint(folderKey, relPoint, screenPoint)에 전달합니다.

기술적 서술 예시

사용자 입력 좌표 환산부는 정방향 배치와 동일한 파라미터 집합에 기초하여 화면 입력 좌표를 원본 이미지 좌표로 환산하고, 상기 환산 결과를 기준 참조점 갱신에 이용한다.

쉬운 설명

화면에서 누른 점이 원래 그림의 어디였는지를 같은 계산법으로 되돌려 찾는다는 뜻입니다.

4.4 wheel zoom / pan / alt-rotate

- wheel zoom: Pinpoint 에서는 현재 local scale 과 global scale 을 읽고, nextGlobalScale 을 계산한 뒤 refScreenX/refScreenY 도 커서 기준으로 함께 보정합니다.
- pan: Pinpoint 에서는 드래그 시 viewport.refScreenX/refScreenY 를 갱신하고, Compare/Analysis 에서는 viewport.cx/cy 를 갱신합니다.
- rotate: Pinpoint 에서는 Alt-drag 시 viewer 별 pinpointRotation[folderKey]를 갱신합니다. 전역 회전은 별도 UI/상태로 존재합니다.

이 부분은 특허 효과 설명에서 매우 중요합니다. 단순히 scale 과 rotation 이 있다는 사실보다, zoom 시 refScreen 도 같이 보정되어 기준점 문맥이 덜 흔들리고, pan 이 image 중심이 아니라 공통 기준 화면좌표를 움직이는 방식으로 정의된다는 점이 Pinpoint 의 작업감을 만듭니다.

4.5 rect-zoom 과 leveling

rect-zoom 역시 Pinpoint 좌표 모델과 연결되어 있습니다. finalizeRectZoom()는 현재 refPoint/refScreen/rotation/scale 을 바탕으로 사각형 중심을 image space 로 역환산하고, 그 지점을 새 refPoint 로 설정한 뒤 개별 scale 을 조정합니다. 또한 leveling 기능은 startLeveling / addLevelingPoint 를 통해 두 점 사이 각도를 계산하고, Pinpoint 에서는 target viewer 개별 회전 또는 global rotation 을 보정합니다.

쉬운 해설: 네모 박스로 확대하는 기능도 결국 “어느 지점을 새 기준으로 삼을지”를 다시 정하는 동작이고, leveling 은 두 점을 찍어 기울어진 이미지를 바로 세우는 기능입니다. 둘 다 특허의 종속항 후보로 넣을 가치가 있습니다.

4.6 동일 파일 다중 viewer 배치

PinpointMode 에는 getFileViewerKeys(file, sourceKey)라는 헬퍼가 있고, file list 항목에서는 Object.values(pinpointImages)를 돌며 동일 File 객체가 어느 viewer 들에 표시되는지를 표시합니다. 즉 현재

구현은 동일 파일이 여러 viewer 에 동시에 배치되는 경우를 고려하고 있으며, 단일 이미지의 여러 refPoint 를 병렬 관찰하는 워크플로와 잘 맞습니다.

이 점은 이전 특허 정리서의 “단일 이미지 다중 포인트 병렬 관찰” 시나리오를 한 단계 더 강하게 뒷받침합니다. 단순 아이디어가 아니라 코드상 고려된 사용 흐름입니다.

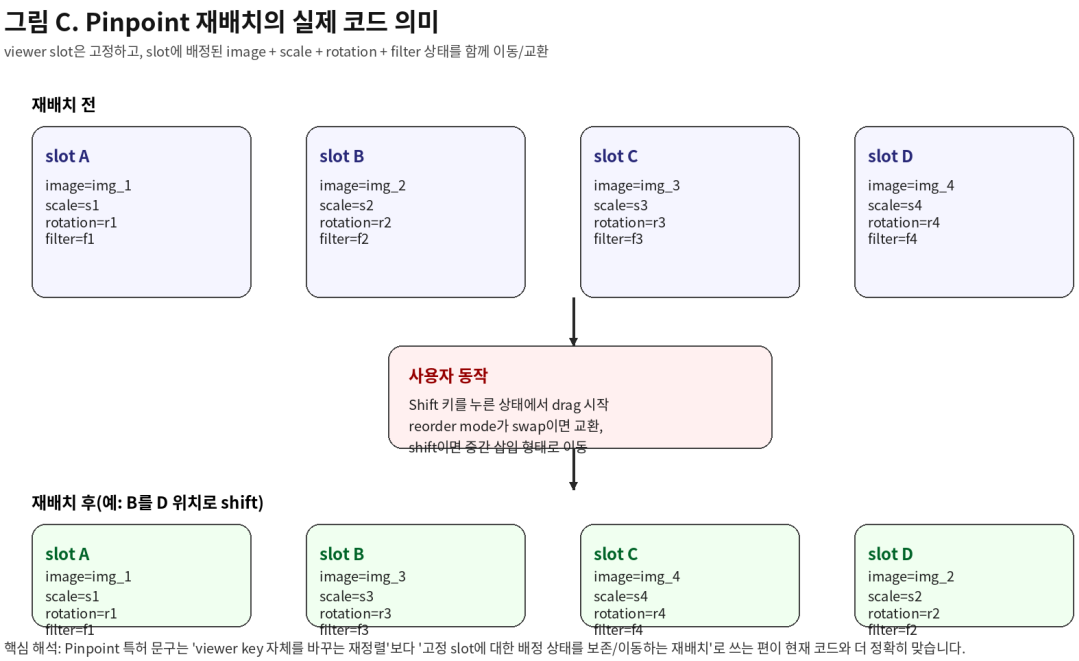

그림 C. Pinpoint 재배치가 실제로 보존하는 상태

4.7 재배치의 실제 의미

Pinpoint 의 재배치는 일반적인 viewer key 재정렬과 다릅니다. DraggableViewer 는 Shift 를 누른 상태에서만 drag 를 시작하고, PinpointMode 는 “viewer slots fixed, reorder only assigned images across fixed slots”라는 주석과 함께 custom onReorder 로직을 사용합니다. 이 로직은 orderedKeys 를 기준으로 imagesByPos, scalesByPos, rotationsByPos, filtersByPos, paramsByPos 를 만들고, shift/swap 규칙에 따라 이동한 뒤 flushSync 안에서 local pinpointImages 와 store 내 transform/filter 상태를 함께 다시 씁니다.

기술적 서술 예시

재배치 제어부는 고정된 표시 슬롯 구조를 유지한 상태에서 상기 슬롯들에 배정된 이미지와 변환 파라미터 및 필터 파라미터를 함께 이동 또는 교환함으로써, 재배치 후에도 정합 문맥을 보존한다.

쉬운 설명

창 자체를 바꾸는 게 아니라 “그 창에 들어 있는 내용물과 그 상태”를 통째로 옮긴다고 보시면 됩니다.

5. 이전 특허 정리서에 대한 팩트체크

아래 표는 이전 작성본의 핵심 진술을 기준으로 “코드로 명확히 확인되는지”, “표현은 맞지만 더 정확히 고쳐야 하는지”, “현재 구현에서는 주장하면 과한지”를 정리한 것입니다.

기존 진술	판정	코드 기반 해석	문서 수정 권고
A 축/B 축 2 축 구조	✅ 확인	Pinpoint 좌표 엔진과 운영 시스템을 나누는 프레임은 실제 코드 구조와 잘 맞습니다.	유지 가능. 다만 A 축은 ImageCanvas + PinpointMode + store 결합으로 쓰는 편이 정확함.
최대 24 viewer	✅ 확인	LayoutGridSelector 에 MAX_VIEWERS = 24 가 있고 README 도 6×4 를 명시합니다.	그대로 사용 가능. 청구항은 “복수 viewer”, 실시예는 24 로 쓰는 것이 좋음.
viewer 별 refPoint 저장	✅ 확인	PinpointMode local state	“창별 기준 참조점 저장부”로 일

		pinpointImages[targetKey].refPoint 에 저장됩니다.	반화 가능. 단, 현 구현은 전역 store 가 아니라 모드 로컬 state.
공통 refScreen 공유	✅ 확인	viewport.refScreenX/refScreenY 를 handleSetRefPoint 와 pan/zoom 이 공통으로 갱신합니다.	그대로 사용 가능. 용어는 "공통 화면 기준좌표" 권장.
정방향/역변환 페루프	✅ 확인	ImageCanvas 의 draw 수식과 click inverse mapping 이 같은 파라미터 집합을 사용합니다.	기존보다 더 강하게 주장 가능. 핵심 청구항 포인트로 승격 권장.
shared viewport 동시 Pan	▲ 표현 조정	현 코드의 정확한 상태명은 Pinpoint 의 refScreen 공유, Compare/Analysis 의 cx/cy 공유입니다.	"shared viewport" 대신 "공통 refScreen 기반 동시 이동" 등으로 수정 권장.
Shift/Swap 재배치	✅ 확인	UI 버튼 + store 모드 + Shift-held drag + custom reorder 가 확인됩니다.	유지 가능. "Shift 키 드래그 + shift/swap 정책"으로 더 구체화하면 좋음.
재배치 후 transform/filter 보존	✅ 확인	PinpointMode custom reorder 에서 images/scales/rotations/filters/params 를 함께 재배치합니다.	이전 문서보다 더 강하게 강조 가능. 차별 포인트로 추천.
단일 이미지 다중 포인트 병렬 관찰	✅ 확인	동일 File 객체가 여러 viewer 에 표시되는 경우를 UI 가 추적합니다.	실시예로 자신 있게 유지 가능.
자동 정합/자동 특징점 정렬	❌ 미확인	현재 코드에는 자동 특징점 기반 registration 파이프라인이 없습니다.	절대 중심 문구로 쓰지 말 것. "사용자 주도 인터랙티브 정합"으로 수정.
Analysis 모드 slot 별 독립 rotation	❌ 미확인	현재 구현은 rotation={analysisRotation}으로 공유 회전입니다.	Pinpoint 특허 본문과 섞지 말고, 제품 설명에서만 주의 깊게 언급.
완전한 client-only / 외부 통신 없음	◐ 부분 확인	이미지 처리 파이프라인은 로컬 중심이지만 logging/update endpoint 가 존재합니다.	"민감 이미지 처리 경로는 로컬 중심" 정도로 낮춰 쓰는 편이 안전.
Compare filename 자동 매칭	✅ 확인	matchFilenames(activeFolders, stripExt) 호출이 존재합니다.	특허 핵심은 아니나 제품 동작 설명에는 사용 가능.
폴더 변경 자동 감지	✅ 확인	picker 5 초 sync 와 Electron fs.watch 경로가 모두 존재합니다.	운영 시스템 부가 기능으로만 언급 권장.
leveling 도구	✅ 확인	두 점 기반 각도 계산 후 individual/global rotation 수정이 존재합니다.	종속항 후보 또는 보조 실시예로 추가 권장.

6. 특허 문서에 반영해야 할 보강 포인트

6.1 독립항 중심축 재정의

- 독립항 A: viewer 별 refPoint 저장 + 공통 refScreen 저장 + local/global transform 결합 + 정방향 배치 + 입력 좌표 역환산
- 독립항 B: layout 선택 + Pin/Pan 분리 + fixed-slot 재배치 + 상태 보존 + 동기 이동
- 독립항 C(선택): 동일 이미지 다중 viewer 배치를 통한 다중 refPoint 병렬 관찰 또는 두 점 leveling 보정

기존 문서는 A 축/B 축 분리 아이디어가 좋았지만, 이번 코드 분석을 반영하면 A 축은 "수학식"보다 "정방향 배치와 역방향 환산의 동일 파라미터 페루프"가 더 강한 논점이고, B 축은 "재배치 후 상태 보존"이 단순 UI 편의가 아니라 실제 기술 효과를 만드는 구성이란 점이 더 선명해집니다.

6.2 종속항 후보로 추천되는 세부 구현

- wheel zoom 시 기준 화면 좌표(refScreen)를 커서 기준으로 함께 보정하는 구성
- rect-zoom 사각형 중심을 image space 로 역환산해 새 refPoint 로 채택하는 구성
- Shift 키 기반 drag 와 shift/swap 정책을 결합한 재배치 구성
- 재배치 시 image + scale + rotation + filter + filter parameter 를 함께 이동시키는 구성
- 두 점 leveling 에 의해 individual rotation 또는 global rotation 을 갱신하는 구성
- 동일 이미지의 복수 배치를 허용하고, viewer 별 서로 다른 refPoint 를 유지하는 구성

6.3 반드시 피해야 할 과장 표현

피해야 할 표현	이유	권장 대체 문구
완전 자동 정합	현재 코드에는 자동 특징점 registration 이 없음	사용자 주도 정밀 정합 / 인터랙티브 정합
shared viewport(포괄적)	Pinpoint 와 Compare/Analysis 의 공유 상태가 다름	공통 화면 기준좌표(refScreen) 공유 / 중심 viewport 공유
viewer 별 refPoint 가 전역 store 에 저장됨	현 구현은 PinpointMode local state	표시창별 기준 참조점 저장부에 저장됨
모든 처리와 통신이 완전 로컬	usage logging, updater 경로가 존재	이미지 처리 경로는 로컬 중심으로 수행됨
Analysis 모드도 per-slot 독립 rotation	main 브랜치 현재 코드와 불일치	slot 별 필터는 독립, rotation 은 현재 공유

7. 코드 근거 맵

아래 위치는 특허 보강에 직접 연결되는 핵심 코드 포인트입니다. 변리사 전달용 부록에는 이 표를 그대로 넣거나, 파일/라인만 간략화한 버전을 사용하셔도 됩니다.

파일/라인	확인 내용
README.md: 1-9	tri-mode workspace, compare/pinpoint/analysis 개요, local-first 설명, 24 viewer, Pinpoint ref pin/scale/rotation/leveling
package.json: dependencies / usageLogging / deployTargets	compareX 패키지명, Electron/Vite/React/Zustand/OpenCV/UTIF, logging/update endpoint 설정
src/App.tsx: 1201-1248	Mode selector, LayoutGridSelector, Pin/Pan selector, Shift/Swap button, global rotation/scale UI
src/store.ts: 391-423	viewport(refScreenX/Y), pinpointScales, pinpointGlobalScale, pinpointRotations, pinpointGlobalRotation default state
src/store.ts: 487-515	setPinpointMouseMode, setPinpointReorderMode, setViewport, setPinpointScale/GlobalScale, setPinpointRotation/GlobalRotation
src/store.ts: 1000-1055	leveling state 와 두 점 기반 angle 계산 및 rotation 반영
LayoutGridSelector.tsx: 27-30, 61-64	MAX_VIEWERS = 24, rows×cols <= 24 제약
DraggableViewer.tsx: 32-95	Shift key 가 눌린 drag 에서만 reorder 시작 및 drop 처리
PinpointMode.tsx: 478-488	handleSetRefPoint 가 refPoint 와 refScreenX/Y 를 함께 갱신
PinpointMode.tsx: 507-542	loadFileToViewer 가 file+refPoint+sourceKey 를 viewer 에 로드
PinpointMode.tsx: 452-460, 850-855	동일 파일이 어느 viewer 에 표시되는지 추적
PinpointMode.tsx: 938-1016	slot 고정형 custom reorder + flushSync + transform/filter 동반 이동
ImageCanvas.tsx: 622-652	Pinpoint 정방향 배치 수식(drawX/drawY)
ImageCanvas.tsx: 1045-1095	click inverse mapping(screen→image) 및 onSetRefPoint 호출
ImageCanvas.tsx: 1129-1199	rect-zoom finalize 와 새 refPoint/scale 설정
ImageCanvas.tsx: 1284-1293	wheel zoom 시 refScreen 동반 보정
ImageCanvas.tsx: 1361-1463	Pinpoint pan/alt-rotate 처리
CompareMode.tsx: 349-367	primary viewer 및 matchFilenames 기반 compare flow
AnalysisMode.tsx: 671-682	analysisFile 공통 사용, slot 별 filters, 공유 analysisRotation
useUsageLogging.ts / usageLogger.ts / electron.js / preload.js	APP_LAUNCH usage log, endpoint POST, updater IPC, folder watch IPC

8. 최종 결론

GitHub main 브랜치의 실제 코드만 놓고 보아도, Pinpoint 는 특허 문서로 정리할 충분한 구조적 개성을 갖고 있습니다. 특히 "viewer 별 refPoint + 공통 refScreen + 정방향/역방향 페루프 + fixed-slot reorder with state preservation"의 결합은 단순 sync viewer 나 단순 manual zoom/pan 도구보다 한 단계 정교한 작업 체계를 보여 줍니다.

따라서 다음 단계에서는 기존 특허 정리서를 자동 정렬 중심으로 넓히기보다, 현재 구현된 인터랙티브 정합 엔진과 운영 시스템을 가장 정확하게 드러내는 방향으로 문장을 다듬는 것이 좋습니다. 아래 별도 첨부하는 보충 정리서는 바로 그 목적을 위해 작성했습니다.